Team handoff - Online Cab Booking System

GitHub and branches

Repository: https://github.com/KevinSamson/onlinecabbooking.git

Person	Feature branch
P1	`feature/p1-auth-admin`
P2	Not yet recorded in the repository; confirm with P2
P3	Not yet recorded in the repository; confirm with P3
P4	Not yet recorded in the repository; confirm with P4

After the shared foundation is pushed, clone with `git clone https://github.com/KevinSamson/onlinecabbooking.git`, then `cd onlinecabbooking`. Switch with `git switch YOUR_ASSIGNED_BRANCH` (or `git switch -c YOUR_ASSIGNED_BRANCH` if it has not been created). Commit your own work and run `git push -u origin YOUR_ASSIGNED_BRANCH`. Do not push to another person's branch.

Local database and environment

Database name: `online_cab_booking`. Each teammate uses **their own local MySQL installation and credentials**. Import the shared tables and demo data from the project root:

```
mysql -u YOUR_LOCAL_MYSQL_USER -p < schema.sql
mysql -u YOUR_LOCAL_MYSQL_USER -p online_cab_booking < seed.sql
```

Create your own `.env` from `.env.example` and replace placeholders with your local database credentials and your own JWT secret. Nobody needs P1's MySQL root password, `app_user` password, or `.env` file.

```
PORT=3000
HOST=127.0.0.1
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=app_user
DB_PASSWORD=replace_me
DB_NAME=online_cab_booking
JWT_SECRET=replace_with_a_long_random_secret_at_least_32_characters
JWT_EXPIRES_IN=12h
```

Demo website logins

These are **website accounts from** `seed.sql`, not MySQL usernames or passwords. All four use the local demo website password `DemoPass!2026`.

Role	Website email
Admin	`admin@astercab.test`
Driver	`driver@astercab.test`
Passenger	`passenger@astercab.test`
Passenger	`passenger2@astercab.test`

Shared tables

Table	Purpose
`users`	Passenger, driver, and admin accounts and access status.
`driver_verifications`	Driver identity/licence files and review decisions.
`vehicles`	Driver vehicles and admin authorization status.
`rides`	Shared booking and trip record for P2 and P3.
`ride_status_history`	Audit trail of ride status changes.
`ride_locations`	Driver location samples for live tracking.
`ride_schedules`	Scheduled ride activation details.
`fare_estimates`	Fare comparison estimates.
`emergency_contacts`	Passenger-owned trusted contacts.
`sos_alerts`	Ride safety alerts.
`fare_splits`	Split-fare header and settlement status.
`fare_split_participants`	Each person's share and payment status.
`lost_found_reports`	Lost item reports tied to rides.

Ownership and integration

- **P1:** shared backend/schema, authentication, driver verification, admin management, emergency contact CRUD.
- **P2:** passenger booking, fares, scheduling, ride history.
- **P3:** driver portal, vehicles, requests, trip handling.
- **P4:** tracking, SOS, fare splitting, route deviation, Lost & Found.

`schema.sql` is shared and owned by P1; coordinate changes instead of independently overwriting it. P2 creates rides and P3 transitions them, so agree on `rides` fields and status rules. P4 may read `emergency_contacts` only for the authorized ride passenger when handling SOS. P3 should require an approved `driver_verifications.status` and approved `vehicles.status` before ride acceptance.

Essential existing APIs (all under `/api`): `POST /auth/register`, `POST /auth/login`, `GET /auth/me`; driver `GET /verification/me`, `POST /verification` (multipart documents); passenger `GET/POST /emergency-contacts`, `PUT/DELETE /emergency-contacts/:id`; admin `GET /admin/drivers`, `GET /admin/passengers`, `PATCH /admin/users/:id/status`, `GET /admin/vehicles`, `PATCH /admin/vehicles/:id/status`, `GET /admin/verifications`, and `PATCH /admin/verifications/:id`. Protected routes require `Authorization: Bearer <JWT>`; successful JSON uses `{ "success": true, "message": "...", "data": {} }` and failures use `{ "success": false, "message": "..." }`.